

주다훤 (Dahwon Ju) 이력서

Frontend Engineer

전화번호 | 010-6518-8253
이메일 | hwondaa@gmail.com
링크 | [Github](#) [Blog](#)

동료의 10분을 줄일 수 있다면, 저의 30분이 아깝지 않습니다.
팀의 병목현상을 기술로 해결하고, 소통 비용을 줄이는 프로세스를 만드는 일에 보람을 느낍니다.
문제가 명확해질수록 더 깊이 몰입하며, 한정된 리소스 안에서 현실적인 대안을 찾아 프로젝트를 끝까지 완수하는 과정에서 가장 큰 성취감을 느낍니다.

경력

(주)한국해양기상기술

Frontend Developer

2023.01 ~ 현재 (3년 3개월~)

(주)한국해양기상기술은 기상청, 국립수산물과학원 등 주요 공공기관을 대상으로 지도 기반의 해양·기상 데이터 시각화 및 연구 지원 플랫폼을 구축하는 해양·기상 IT 전문 기업입니다.

주요 성과

- 고객사 이탈 리스크가 있는 상황에서 UI/UX 개선 TF팀 운영으로 사용자 만족도 2배 개선
- LLM 기반 자동 코드 리뷰 시스템 구축으로 리뷰 효율 50% 향상
- 사내 공용 UI 라이브러리(@koast/ui) 구축으로 개발 리소스 최대 98% 절감
- 프로젝트 PM 및 UI/UX 설계 경험

업무 경험

기상청 서비스

2023.04 ~ 2025.08 (2년 4개월)

지도 기반 해양정보를 제공하는 대국민 서비스 **해양정보기상포털** 및 기상청 내부 **솔루션** 개발 및 유지보수

1. 메인페이지 만족도 4점 → 8점 - 서비스 첫인상을 다시 설계

(3인 팀, 팀 리드·Frontend 담당)

요구사항이 적체된 상황에서 **대국민 조사 결과 메인페이지 UI 만족도까지 낮게 나타나**, 프로젝트 신뢰를 회복하지 못하면 **고객사(기상청) 핵심 프로젝트를 잃을 수 있는 상황**이었습니다. 이에 기상청 실무진과 직접 커뮤니케이션하며, 사용자 불편을 구조적으로 정의하고 단기간 내 개선안을 제품에 반영하는 UI/UX 개선 TF를 주도했습니다.

- 약 300명 규모의 설문을 기반으로, 서비스 만족도가 가장 낮은 메인페이지 UI를 개선할 TF 구성
- Figma 기반으로 **A·B·C·D 개선안과 간단한 인터랙션 와이어프레임 설계**로, 회의 단계에서 빠른 의사결정 지원
- 저대비 색상과 분산된 메뉴 구조로 인해 주요 기능이 노출되지 않던 문제를, **좌측 고정형 메뉴 + 썸네일 기반 구조로 재설계해 가시성과 탐색성을 개선**
- 동일하거나 의미가 불명확한 **state**, 심한 **props drilling** 등 복잡해진 상태 관리 구조를 **Vuex로 재설계**

→ 4개월 내 PC/Mobile 메인페이지 전반 UI 개편 완료

→ 네이버 폼 설문(응답 672명) 기준, **사용자 만족도 2배 개선(4점→8점)**

→ 배포 후 1개월 전체 사용자 로그 기준, **평균 인터랙션 시간 2배 증가(3분 → 6분)**

2. “코드리뷰 3시간이 너무 힘들어요.” - 로컬 LLM 기반 자동 코드 리뷰 시스템 구축

(단독 개발)

보안 정책상 외부 클라우드를 사용할 수 없는 환경에서, 오타자·코드 컨벤션 위반·반복적인 검수 항목까지 시니어 엔지니어가 수동으로 검수하고 있었습니다. 또한 리뷰어가 변경 맥락을 파악하는 데 건당 30분 이상이 소요되어 코드리뷰 병목이 발생하고 있었고, 반복적이고 정형화된 검수 항목을 1차적으로 자동화할 필요가 있었습니다.

- 내부 NAS 환경에 Ollama 기반 로컬 LLM 서버를 설계·구축
- Systemd 워커와 GitLab 파이프라인을 연동해 MR 단위 실시간 자동 리뷰 프로세스 구현
- 리뷰가 누락된 MR을 자동 식별하고, 반복적·정형화된 검수 항목을 1차 자동 점검하도록 설계

→ 팀원 5명 사용 기준, 리뷰 효율 50% 향상(3시간→1.5시간)

→ 컨벤션 위반 사항 사전 필터링으로 휴먼 에러 방지 및 품질 일관성 유지

3. 누적된 FE 기술 부채 해소 및 고객사 요구사항 대응 구조 개선

(5인 팀, Frontend 담당)

외주 중심으로 누적된 기술 부채로 요구사항 30여 건이 장기간 적체되었고, 신뢰도도 함께 저하되고 있었습니다. 복불 중심으로 확장된 구조 탓에 메뉴 상태와 로직 흐름 파악이 어려워 수정 영향 범위 예측도 힘들었습니다. 이대로는 안정적인 서비스가 어렵다고 판단해, 컴포넌트 구조·타입 체계·상태 관리 순으로 프론트엔드를 재정비했습니다.

- 2천 줄 이상 비대해진 Vue 컴포넌트에 script setup을 도입하고, 데이터 가공·이벤트 처리·API 통신 로직을 분리해 화면 렌더링 중심으로 단순화
- Props 타입 외 대부분이 as any에 의존하던 상태를 개선하기 위해, src/types 구조를 만들고 메뉴별 타입과 공통 인터페이스를 분리·확장하는 방식으로 정리
- 대용량 시계열 데이터는 깊은 반응성 비용이 커서 ref, reactive 사용 시 갱신마다 400~500ms까지 지연이 발생했으며, 이를 shallowRef로 전환해 버튼 클릭 시 전체 값 교체 방식으로 최적화

→ 누적된 요구사항 100% 완료, 타임 슬라이더의 상태 갱신 지연과 자잘한 버그 획기적 감소

스마트양식장 연구 지원 플랫폼 개발

2025.06 ~ 2025.12 (6개월)

양식장 운영과 양식 데이터 시뮬레이션 기능을 통합한 연구 데이터 분석 플랫폼 구축

(2인 팀, PM·UI/UX·Frontend 담당)

1. Vue2 → React 마이그레이션 및 공통 화면 구조 재설계

지원 종료된 Vue2 기반 서비스는 Table 중심 화면이 많았지만, 유사한 화면도 구현 방식이 제각각이어서 유지보수와 기능 확장이 어려웠습니다. 이에 조직 표준인 React + TailwindCSS와 @koast/ui 도입 방향에 맞춰, 단순 이관이 아닌 확장 가능한 구조로 전면 재설계했습니다.

- 반복 화면을 공통 Wrapper 구조로 통일하고, Table UI는 컬럼 비율·정렬 여부 등을 props로 제어하도록 설계
- 공통 컴포넌트와 화면 컴포넌트의 책임을 분리해, UI 재사용성과 화면별 확장성을 함께 확보

→ 반복 화면 생산성이 개선되어, 일부 페이지는 하루 2~3개 화면 개발 가능

2. 데이터 분석 소요 시간 최대 95% 단축 - 차트 라이브러리 선택부터 최적화까지

연구진은 과거 실측 데이터와 미래 예측 데이터를 하나의 흐름으로 비교해 보고 싶어 했지만, 기존 방식으로는 이를 자연스럽게 시각화하기 어려웠습니다. 또한 초기에는 입력값이 바뀔 때마다 백엔드 계산 결과를 받아 차트를 다시 그리는 구조여서, 100~200ms 수준의 지연이 누적되며 화면이 버벅이는 문제가 있었습니다.

- Recharts를 활용해 과거 실측 데이터와 미래 예측 데이터를 하나의 차트로 보여주는 성장 예측 시각화 구현
- 시뮬레이션 계산 로직을 프론트엔드로 이전하고, 입력값에 디바운싱을 적용해 차트 리렌더링 부담 완화
- 최종 결과 저장 및 엑셀 다운로드 시점에만 백엔드와 연동하도록 처리 흐름 분리

→ 실사용 연구진 3명 기준, 데이터 분석 소요 시간 약 40분 → 2분 수준으로 단축

→ 연구진의 만족도가 높았던 핵심 기능으로 평가받음

3. 화면을 만들기 전에, 요구사항 먼저 보이도록

연구진의 요구사항은 “보고 싶은 데이터는 있는데 어떤 화면이 가장 적합한지”가 명확하지 않은 경우가 많았습니다. 요구가 추상적인 상태에서 바로 구현에 들어갈 경우 재작업 위험이 높다고 판단해, 빠르게 시안을 제시하며 요구사항을 구체화하는 방식으로 의사결정을 지원했습니다.

- Figma로 여러 화면 시안을 제작해 연구진이 원하는 데이터 표현 방식과 사용 흐름을 빠르게 구체화
- JIRA 기반 WBS 작성으로 기능의 UI 방향성과 우선순위 정리
- 연구 환경과 태블릿 사용성을 고려해 다크 모드를 선제적으로 적용

→ 불명확한 요구사항을 빠르게 구체화해 재작업 위험 감소, 제한된 리소스 환경에서도 일정 내 프로젝트 완료

@koast/ui

2025.03 ~ 2025.06 (3개월)

React, TailwindCSS 기반 사내 공용 UI 라이브러리 구축

(단독 개발)

Thymeleaf, Vue 등 여러 스택으로 제작된 기존 프로젝트 환경에서 프론트엔드 작업이 특정 인력에 집중되며 UI 구현 병목이 반복적으로 발생하고 있었습니다. 이를 해결하기 위해 React, TailwindCSS 기반의 표준 스택을 제안하고, 전사 서비스에서 재사용 가능한 공용 UI 라이브러리 @koast/ui를 자발적으로 구축했습니다.

- CTO와 논의를 거쳐 React, TailwindCSS 기반의 프론트엔드 표준 스택 도입
- 도메인 특화 공통 컴포넌트를 포함한 사내 React UI 라이브러리 설계·구축
- GitHub Actions 기반 Build-Release-Deploy 파이프라인을 자동화해 배포 효율 개선
- Storybook 기반 문서화를 구축해 컴포넌트 사용성과 협업 편의성 향상

→ 복잡한 시각화 컴포넌트(Time Slider) UI 구현 시간을 8시간 → 10분 이내로 단축

사이드 프로젝트

React-multi-email

Github Stars 300+, Weekly Downloads 10만+의 리액트 기반 멀티 이메일 입력폼 라이브러리입니다. 실사용 이슈를 분석해 버그를 수정하고, 테스트와 문서화를 보강하며 라이브러리 신뢰도를 높이는 데 기여했습니다.

- onBlur, disabled 등 주요 시나리오 중심의 Jest 유닛 테스트 작성으로 핵심 로직 안정성 강화
- 이메일 입력 시 validateEmail이 false를 반환하거나 오류가 발생하면 스피너가 종료되지 않는 Bug Fix

Diki

AI 초심자를 위한 데이터 용어사전(Data Wiki)으로, 정적 빌드 기반으로 비용을 최소화하면서도 검색 정확도를 높인 용어사전 서비스입니다.

- SSG 기반 구조로 무료 티어 환경에서 운영 가능한 비용 효율적 아키텍처 설계
- lunr.js를 커스터마이징해 제목·태그 가중치, 접두사/접미사, Fuzzy 검색을 지원하는 검색 구조 구현

ZIZIZIP

파편화된 공공임대 공고 정보를 지도 기반으로 탐색할 수 있도록 구현한 서비스입니다. 사용자 피드백을 바탕으로 기능을 개선하며 실제 사용성을 검증했습니다.

- 기관별로 다른 엑셀 컬럼명을 표준 인터페이스로 변환하는 데이터 전처리 로직 구현
- Web Worker, 줌 레벨 기반 클러스터링 전환을 적용해 대용량 지도 렌더링 성능 최적화

→ 여러 부동산 커뮤니티 사용자 피드백을 반영해 서비스 개선, LH 공고일 기준 최대 활성 사용자 578명 기록